

《数据结构》课程主要内容

二叉树遍历算法 (Binary Tree Traversal)

二叉树遍历是数据结构中非常重要的基础操作之一。所谓遍历 (Traversal)，是指按照某种特定的顺序访问二叉树中的每一个节点，并且保证每个节点只被访问一次。通过不同的访问顺序，可以实现不同的数据处理逻辑。

二叉树的遍历方式主要分为两大类：

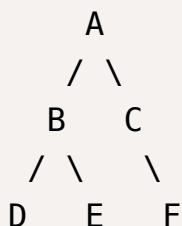
- **深度优先遍历 (DFS, Depth First Search) **
- **广度优先遍历 (BFS, Breadth First Search) **

其中最常见的三种遍历方式属于深度优先遍历：

1. 前序遍历 (Preorder Traversal)
2. 中序遍历 (Inorder Traversal)
3. 后序遍历 (Postorder Traversal)

一、二叉树结构示例

假设存在如下二叉树结构：



该二叉树包含 6 个节点，每个节点最多有两个子节点，分别为：

- 左子节点 (Left Child)

- 右子节点 (Right Child)

二、前序遍历 (Preorder Traversal)

1 遍历规则

前序遍历的访问顺序为：

根节点 → 左子树 → 右子树

即：

1. 先访问当前节点
2. 再递归遍历左子树
3. 最后递归遍历右子树

2 遍历过程

以前面的二叉树为例：

Step1: 访问 A

Step2: 遍历 A 的左子树

Step3: 访问 B

Step4: 访问 D

Step5: 访问 E

Step6: 遍历 A 的右子树

Step7: 访问 C

Step8: 访问 F

最终结果：

A → B → D → E → C → F

3 Python递归实现

```
```python
class TreeNode:
 def __init__(self, val):
 self.val = val
 self.left = None
 self.right = None

def preorder(root):
 if root is None:
 return

 print(root.val, end=" ")
 preorder(root.left)
 preorder(root.right)
```
```

三、中序遍历 (Inorder Traversal)

1 遍历规则

中序遍历的访问顺序为：

左子树 → 根节点 → 右子树

即：

1. 先遍历左子树
2. 再访问当前节点
3. 最后遍历右子树

2 遍历过程

Step1: 遍历 A 的左子树
Step2: 访问 D
Step3: 访问 B
Step4: 访问 E
Step5: 访问 A
Step6: 遍历 A 的右子树
Step7: 访问 C
Step8: 访问 F

最终结果:

D → B → E → A → C → F

3 Python递归实现

```
def inorder(root):  
    if root is None:  
        return  
  
    inorder(root.left)  
    print(root.val, end=" ")  
    inorder(root.right)
```

四、后序遍历 (Postorder Traversal)

1 遍历规则

后序遍历的访问顺序为:

左子树 → 右子树 → 根节点

即:

1. 先遍历左子树
2. 再遍历右子树

3. 最后访问当前节点

2 遍历过程

```
Step1: 遍历 A 的左子树
Step2: 访问 D
Step3: 访问 E
Step4: 访问 B
Step5: 遍历 A 的右子树
Step6: 访问 F
Step7: 访问 C
Step8: 访问 A
```

最终结果：

```
D → E → B → F → C → A
```

3 Python递归实现

```
def postorder(root):
    if root is None:
        return

    postorder(root.left)
    postorder(root.right)
    print(root.val, end=" ")
```

五、层序遍历（Level Order Traversal）

层序遍历属于 **广度优先搜索（BFS）**，按照树的层级从上到下、从左到右依次访问节点。

遍历顺序：

```
A → B → C → D → E → F
```

实现层序遍历通常需要借助 **队列（Queue）** 数据结构。

Python实现

```
from collections import deque

def level_order(root):
    if root is None:
        return

    queue = deque([root])

    while queue:
        node = queue.popleft()
        print(node.val, end=" ")

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)
```

六、时间复杂度与空间复杂度

| 遍历方式 | 时间复杂度 | 空间复杂度 |
|------|--------|--------|
| 前序遍历 | $O(n)$ | $O(h)$ |
| 中序遍历 | $O(n)$ | $O(h)$ |
| 后序遍历 | $O(n)$ | $O(h)$ |
| 层序遍历 | $O(n)$ | $O(n)$ |

说明：

- n 表示节点总数
- h 表示树的高度
- 递归遍历需要使用函数调用栈，因此空间复杂度与树的高度有关

七、遍历方式总结

| 遍历类型 | 访问顺序 | 特点 |
|------|-----------|-------------|
| 前序遍历 | 根 → 左 → 右 | 常用于复制树结构 |
| 中序遍历 | 左 → 根 → 右 | 二叉搜索树得到有序序列 |
| 后序遍历 | 左 → 右 → 根 | 常用于删除树 |
| 层序遍历 | 按层访问 | 常用于最短路径问题 |

记忆口诀：

前序：根左右
中序：左根右
后序：左右根

八、应用场景

二叉树遍历在计算机科学中具有广泛的应用，例如：

- 1. 表达式求值
- 2. 编译器语法树处理
- 3. 文件系统结构
- 4. 数据库索引结构
- 5. 人工智能搜索算法

例如，在 **二叉搜索树（BST）** 中，中序遍历可以得到一个 **有序序列**，因此在排序和查找算法中具有重要作用。

九、小结

二叉树遍历是理解树结构的基础操作。不同的遍历方式反映了不同的访问策略：

- **前序遍历**：先处理节点，再处理子树
- **中序遍历**：适用于有序结构
- **后序遍历**：先处理子树，再处理节点
- **层序遍历**：按层级逐层访问

熟练掌握这些遍历方法，对于理解更高级的数据结构（如 **AVL树**、**红黑树**、**B树** 等）具有重要意义。

栈 (Stack) 数据结构

栈 (Stack) 是一种 **线性数据结构**，其特点是 **后进先出 (LIFO, Last In First Out)**。

也就是说，最后进入栈的元素会最先被取出。

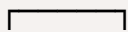
栈在计算机科学中应用非常广泛，例如：

- 函数调用栈
- 表达式求值
- 浏览器前进/后退
- 括号匹配检查
- 深度优先搜索 (DFS)

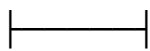
一、栈的基本概念

栈可以理解为一个 **只能在一端进行插入和删除操作的线性表**，这一端被称为 **栈顶 (Top)**，另一端称为 **栈底 (Bottom)**。

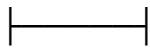
示意图：



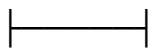
Top → | 5 |



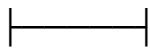
| 4 |



| 3 |



| 2 |



| 1 |
└───┘

Bottom

二、栈的基本操作

栈通常包含以下几种基本操作：

| | |
|----------|----------|
| 操作 | 说明 |
| ----- | ----- |
| push | 向栈顶添加元素 |
| pop | 删除栈顶元素 |
| peek/top | 查看栈顶元素 |
| isEmpty | 判断栈是否为空 |
| size | 获取栈中元素数量 |

三、栈的操作过程示例

假设栈初始为空：

Stack: []

执行以下操作：

push(1)

push(2)

push(3)

栈的状态：

Stack: [1, 2, 3]

执行：

pop()

返回：

3

此时栈变为：

Stack: [1, 2]

四、Python实现栈

在 Python 中，可以使用 ****列表 (list) **** 来实现栈。

1 使用列表实现

```
```python
```

```
stack = []
```

```
入栈
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print(stack) # [10, 20, 30]
```

```
出栈
```

```
stack.pop()
```

```
print(stack) # [10, 20]
```

## 2 封装为栈类

```
class Stack:
 def __init__(self):
 self.items = []

 def push(self, item):
 """入栈"""
 self.items.append(item)

 def pop(self):
 """出栈"""
 if not self.is_empty():
 return self.items.pop()

 def peek(self):
 """查看栈顶元素"""
 if not self.is_empty():
 return self.items[-1]

 def is_empty(self):
 """判断栈是否为空"""
 return len(self.items) == 0

 def size(self):
 """返回栈大小"""
 return len(self.items)
```

使用示例：

```
s = Stack()

s.push(1)
s.push(2)
```

```
s.push(3)

print(s.peek()) # 3
print(s.pop()) # 3
print(s.size()) # 2
```

## 五、时间复杂度分析

操作	时间复杂度
push	$O(1)$
pop	$O(1)$
peek	$O(1)$

由于栈的所有操作都在 **栈顶进行**，因此效率非常高。

## 六、栈的典型应用

### 1 括号匹配问题

判断字符串中的括号是否匹配，例如：

```
()
([{}])
```

算法思路：

1. 遇到左括号入栈
2. 遇到右括号与栈顶匹配
3. 不匹配则错误

示例代码：

```
def is_valid(s):
 stack = []
 mapping = {")": "(", "]": "[", "}": "{"}
```

```
for char in s:
 if char in mapping.values():
 stack.append(char)
 elif char in mapping:
 if not stack or stack.pop() != mapping[char]:
 return False

return len(stack) == 0
```

## 2 浏览器前进后退

浏览器通常使用 **两个栈**：

- 后退栈
- 前进栈

当用户访问新网页时：

当前页面 → 后退栈

点击后退：

当前页面 → 前进栈  
后退栈 → 当前页面

## 七、栈与队列的区别

特性	栈	队列
数据结构	线性结构	线性结构
操作方式	后进先出（LIFO）	先进先出（FIFO）
插入位置	栈顶	队尾
删除位置	栈顶	队头

## 八、小结

栈是一种非常基础但重要的数据结构，其主要特点是：

- **后进先出 (LIFO)**
- 只允许在 **栈顶操作**
- 实现简单但应用广泛

在算法和系统设计中，栈经常用于：

- **递归调用**
- **表达式计算**
- **深度优先搜索**
- **语法解析**